main.py

Visualize

Run ▶

```python
import heapq

GOAL = (1, 2, 3, 4, 5, 6, 7, 8, 0)

def manhattan(state):
    dist = 0
    for i, tile in enumerate(state):
        if tile:
            g = GOAL.index(tile)
            dist += abs(i // 3 - g // 3) + abs(i % 3 - g % 3)
    return dist

def neighbours(state):
    b = state.index(0)
    r, c = divmod(b, 3)
    for dr, dc in ((-1, 0), (1, 0), (0, -1), (0, 1)):
        nr, nc = r + dr, c + dc
        if 0 <= nr < 3 and 0 <= nc < 3:
            n = nr * 3 + nc
```

Output

```
Solved in 2 moves

Step 0
1 2 3
4 _ 6
7 5 8

Step 1
1 2 3
4 5 6
7 _ 8

Step 2
1 2 3
4 5 6
7 8 _
```

Activate Windows
Go to Settings to activate Windows.

```python
def solve(start):
    pq = [(manhattan(start), 0, start, [start])]
    seen = set()
    while pq:
        f, g, state, path = heapq.heappop(pq)
        if state == GOAL:
            return path
        if state in seen:
            continue
        seen.add(state)
        for nxt in neighbours(state):
            if nxt not in seen:
                heapq.heappush(pq, (g + 1 + manhattan(nxt), g + 1,
nxt, path + [nxt]))
    return None

def show(state):
    for i in range(0, 9, 3):
        print(' '.join(str(x) if x else '_' for x in state[i:i + 3]))
```

```
Solved in 2 moves

Step 0
1 2 3
4 _ 6
7 5 8

Step 1
1 2 3
4 5 6
7 _ 8

Step 2
1 2 3
4 5 6
7 8 _
```

Activate Windows
Go to Settings to activate Windows.

```
        seen.add(state)
        for nxt in neighbours(state):
            if nxt not in seen:
                heapq.heappush(pq, (g + 1 + manhattan(nxt), g + 1,
nxt, path + [nxt]))
    return None

def show(state):
    for i in range(0, 9, 3):
        print(' '.join(str(x) if x else '_' for x in state[i:i + 3]))
    print()

start = (1, 2, 3, 4, 0, 6, 7, 5, 8)
path = solve(start)
print("Solved in", len(path) - 1, "moves\n")
for step, s in enumerate(path):
    print("Step", step)
    show(s)
```

```
Solved in 2 moves

Step 0
1 2 3
4 _ 6
7 5 8

Step 1
1 2 3
4 5 6
7 _ 8

Step 2
1 2 3
4 5 6
7 8 _
```